

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	G.Srivardhan	Reg. No.:	192425208
Section / Batch:		Date:	

Instructions to Students

- Answer BOTH scenario-based questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions	Marks
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – SCENARIO BASED ASSESSMENT

A log monitoring system scans large files to detect error patterns using brute-force string matching. a) Explain the working of the algorithm. b) Analyze its time complexity. c) Justify the need for optimized approaches.

Brute-Force String Matching in Log Monitoring Systems

When a log monitoring system scans large log files to detect error messages (such as "ERROR", "FAILED", or "Exception"), it can use the Brute-Force String Matching Algorithm. This algorithm checks every possible position in the log file to determine whether the required pattern exists.

a) Working of the Brute-Force String Matching Algorithm

Algorithm Steps:

1. Let the log file be the text (T) of length n.
2. Let the error keyword be the pattern (P) of length m.
3. Start comparing the pattern with the text from the first character.
4. If all characters match, the pattern is found.
5. If a mismatch occurs, shift the pattern one position to the right and compare again.
6. Continue until all possible positions in the text have been checked.

Example:

- Text (Log): INFOERRORWARN
- Pattern: ERROR

Comparisons:

- Compare with INFOER → Mismatch
- Shift by one position
- Compare with NFOERR → Mismatch
- Shift again
- Compare with ERROR → Match Found

Thus, the algorithm identifies the occurrence of "ERROR" in the log.

b) Time Complexity Analysis

Let:

- n = Length of the log file (text)
- m = Length of the error pattern

Case	Time Complexity	Explanation
Best Case	$O(n)$	Pattern found immediately or first character mismatches at every position.
Average Case	$O(nm)$	Some characters match before mismatches occur.
Worst Case	$O(nm)$	Nearly all characters match before each mismatch, causing maximum comparisons.

Space Complexity: $O(1)$ (constant extra memory)

c) Need for Optimized Approaches

Although brute-force is simple and easy to implement, it becomes inefficient for very large log files because it repeatedly compares the same characters.

Limitations:

- High execution time for gigabyte-sized logs.
- Inefficient when searching for multiple error patterns.
- Repeated character comparisons reduce performance.

Optimized Algorithms:

- Knuth–Morris–Pratt (KMP): Avoids unnecessary re-comparisons using a prefix table.
- Boyer–Moore: Skips several characters after a mismatch, making it faster for long texts.
- Rabin–Karp: Uses hashing for efficient searching, especially for multiple patterns.
- Aho–Corasick: Searches multiple error patterns simultaneously, making it ideal for log monitoring systems.

Conclusion

The Brute-Force String Matching Algorithm is straightforward and suitable for small log files or simple applications. However, for real-world log monitoring systems handling massive volumes of data, optimized algorithms such as KMP, Boyer–Moore, or Aho–Corasick are preferred because they reduce unnecessary comparisons, improve execution speed, and provide better scalability.

A stock market analysis system sorts fluctuating stock prices before generating reports. The data arrives in random order. a) Compare Selection Sort and Bubble Sort for this dataset. b) Analyze their computational cost. c) Recommend the better algorithm with justification.

Stock Market Analysis System: Selection Sort vs Bubble Sort

In a stock market analysis system, fluctuating stock prices arrive in random order and must be sorted before generating reports. Two simple sorting algorithms that can be used are Selection Sort and Bubble Sort.

a) Comparison of Selection Sort and Bubble Sort

Selection Sort

genui{"computing_fundamentals_algorithms_learning_block":{"type_id":"SELECTION_SORT"}}□

- Finds the minimum element from the unsorted portion.
- Swaps it with the first unsorted element.
- Repeats until the entire list is sorted.
- Performs fewer swaps, making it useful when swapping is expensive.

Bubble Sort

genui{"computing_fundamentals_algorithms_learning_block":{"type_id":"BUBBLE_SORT"}}

- Compares adjacent elements.
- Swaps them if they are in the wrong order.
- Repeats multiple passes until no swaps are needed.
- Larger elements "bubble" to the end after each pass.

Comparison Table

Feature	Selection Sort	Bubble Sort
Working Principle	Selects minimum element and places it correctly	Swaps adjacent elements repeatedly
Number of Swaps	Minimum (at most $n-1$)	May perform many swaps
Stability	Not Stable	Stable
Adaptive	No	Yes (optimized version)
Suitable for Random Data Better		Less efficient

b) Computational Cost Analysis

Let n be the number of stock prices.

Case	Selection Sort	Bubble Sort
Best Case	$O(n^2)$	$O(n)$ (optimized)
Average Case	$O(n^2)$	$O(n^2)$

Case	Selection Sort	Bubble Sort
------	----------------	-------------

Worst Case	$O(n^2)$	$O(n^2)$
------------	----------	----------

Space Complexity	$O(1)$	$O(1)$
------------------	--------	--------

Cost Analysis

- Selection Sort
 - Comparisons: Approximately $n(n-1)/2$
 - Swaps: Maximum $n-1$
 - Less data movement.
- Bubble Sort
 - Comparisons: Approximately $n(n-1)/2$
 - Swaps: Can be very large in random datasets.
 - Higher execution time due to frequent swapping.

c) Recommendation with Justification

For a stock market analysis system where prices arrive in random order, Selection Sort is the better choice.

Justification

- Performs significantly fewer swaps than Bubble Sort.
- More efficient for randomly ordered data where many elements are misplaced.
- Requires constant extra memory ($O(1)$).
- Produces better performance in practice because excessive swapping is avoided.

However, for large-scale stock market datasets containing thousands or millions of prices, neither algorithm is ideal. More efficient algorithms such as Merge Sort, Quick Sort, or Heap Sort are preferred because they achieve an average time complexity of $O(n \log n)$.

Conclusion

Although both Selection Sort and Bubble Sort have an average and worst-case time complexity of $O(n^2)$, Selection Sort is recommended for randomly ordered stock price data because it performs far fewer swaps, resulting in lower computational overhead and better overall efficiency. For real-world stock market applications, advanced $O(n \log n)$ sorting algorithms should be used.

Code of Conduct

I G.Srivardhan(192425208)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Question No.	Problem Understanding & Context Analysis (25%)	Application of Concepts (30%)	Analytical & Decision-Making Skills (20%)	Solution Design & Approach (15%)	Clarity, Justification & Presentation (10%)	Total Marks (Each – 50)
Q1	4	3	3	2	4	40
Q2	4	4	4	3	2	46
Overall Total (100)						86

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____